Ream: A Sparse Text Format for LLM Spreadsheet Comprehension

Siqi Chen
Runway Financial, Inc.
siqi@runway.com

Abstract

We introduce **Ream**, a UTF-8, line-oriented text format designed to serialize spreadsheet workbooks for large language model consumption. Ream encodes each row with its absolute row number, uses A1-style column letters for sparse cell addressing, preserves formula text in R1C1 notation, and supports defined names and structured table references. Its sparse design omits blank cells, yielding compact representations that scale to enterprise workbooks without exceeding model context windows.

To validate the design, we benchmark Ream against **10 alternative serialization formats**—CSV, Markdown, HTML, XML, JSON, Pandas, Markdown-KV, Cell-Address Markdown, Reverse-Index Values, and Raw OOXML—on **four evaluation corpora** (FRTR-Bench, SpreadsheetBench, MiMoTable, NL2Formula) across three OpenAI models (GPT-4o-mini, GPT-5.4-nano, GPT-5.4), executing more than **19,000 individual LLM calls**.

On enterprise multi-sheet workbooks, Ream achieves **92.0%** accuracy on GPT-5.4, tying the best format (Cell-Address MD) while using **26% fewer tokens**. On GPT-5.4-nano, Ream leads all formats at **78.5%**. Across all three corpora, Ream achieves the **highest cross-corpus average** (79.0%) and is the **only top-tier format with zero context-length failures** on every model.

1 Introduction

Spreadsheets remain the dominant medium for structured data in business. As LLM-powered analytics pipelines proliferate, a critical bottleneck emerges: workbook content must be serialized into text before a model can reason about it, yet the choice of serialization format has received little systematic study.

Practitioners default to CSV or Markdown. Researchers have proposed HTML [Sui et al., 2024], JSON with cell addresses [Dong et al., 2024], inverted-index compression, and record-oriented layouts—but prior comparisons were limited to flat tables, single tasks, or a handful of formats, and none evaluated on multi-sheet enterprise workbooks.

We present **Ream**, a text format designed from first principles for LLM spreadsheet comprehension, together with the most comprehensive serialization benchmark to date. Our contributions:

1. **The Ream format**: a sparse, line-oriented encoding that combines absolute row numbering, A1-style column addressing, R1C1 formula notation, defined names, and structured table references in a token-efficient layout.
2. **An 11-format benchmark** across four corpora and three models (GPT-4o-mini, GPT-5.4-nano, GPT-5.4), comprising 19,000+ LLM calls.
3. **Two key findings**: (a) format choice matters for complex spreadsheets (30 pp spread) but not for simple tables (4 pp); (b) Ream achieves the best cross-corpus accuracy while using 26–52% fewer tokens than its nearest competitors.

2 The Ream Format

Ream is a UTF-8, line-oriented, sparse text format governed by six rules:

1. Rows are addressed with absolute row numbers as plain line labels.
2. Columns are addressed by position, with optional A1-style prefixes (`B=`, `B:M=`) for sparse gaps.
3. Selectors outside formulas use A1 notation.
4. Formulas use R1C1 notation for cell and range coordinates.
5. Defined names and structured table references are first-class formula operands.
6. Omitted cells are blank.

2.1 Design rationale

Three design choices distinguish Ream from prior formats:

A1-style column letters. Formats that use numeric column indices (`C2=`, `C13=`) require models to map indices to column identity—an in-context learning step. Ream uses the A1 column letters (`B=`, `M=`) that models have encountered extensively in training data from Excel documentation, forum posts, and tutorials.

Sparse row encoding. CSV and Markdown emit every cell in every row, including empty trailing columns. Ream omits blank cells entirely. On a 1,000-row, 26-column workbook where only 6 columns per row contain data, Ream uses roughly half the tokens of CSV.

Structural directives. `#!SHEET`, `#!HEADERS`, `#!TABLE`, and `#!NAME` directives encode multi-sheet boundaries, header rows, named tables, and defined names—structural information that CSV, Markdown, and HTML discard.

2.2 Example

A P&L workbook with a named tax rate, cross-sheet formula references, and metadata:

```
#!REAM 9
#!NAME tax_rate 'Assumptions'!B2
#!SHEET Assumptions
2 | Tax Rate | 0.25 |

#!SHEET "P&L"
#!HEADERS 1:1
#!NAME revenue_inputs B2:M2

1 | B=Jan | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec |
2 | Revenue | 1000 | 1050 | 1100 | 1150 | 1200 | G:M==RC[-1]*1.08 |
3 | COGS | B:M==R[-1]C*0.6 |
4 | Gross Profit | B:M==R[-2]C-R[-1]C |
6 | Headcount | B:C=50 | D:M=52 |
7 | Avg Salary | B:M=8500 |
8 | Payroll | B:M==R[-2]C*R[-1]C |
9 | Rent | B:M=15000 |
10 | Total OpEx | B:M==R[-2]C+R[-1]C |
12 | Tax | B:M==R4C*tax_rate |
13 | Net Income | B:M==R4C-R10C-R[-1]C |

#!FMT 2:4 usd0
#!TAG B2:F2 "sales input"
#!TAG B13:M13 output
```

Table 1: The 11 formats evaluated. "Cell IDs" = explicit cell coordinates.

Format	Family	Cell IDs	Values	Multi-sheet	Provenance
Ream	Cell-addr.	A1-style	✓	`#!SHEET`	This work
CSV	Delimiter	—	✓	text header	Baseline
Markdown	Markup	—	✓	text header	Baseline
HTML	Markup	—	✓	`<h3>` tag	Sui et al. [2024]
XML	Markup	row attr.	✓	`<sheet>` tag	Sui et al. [2024]
JSON	Structured	col. letters	✓	nested object	Baseline
Pandas	Record	row index	✓	section headers	`df.to_string()`
Markdown-KV	Record	—	✓	section headers	ImprovingAgents (2025)
Cell-Address MD	Cell-addr.	A1-style	✓	section headers	Dong et al. [2024]
Reverse-Idx Val	Compressed	ranges	✓	JSON object	This work
Raw OOXML	Native	XML tags	✓	XML files	Baseline

Key features visible in this example: sparse row numbering (row 5 and 11 are blank and omitted), range compaction (`B:M=8500`), addressed formulas with `==`, cross-sheet name references (`tax_rate`), and metadata directives.

3 Related Work

Table serialization for LLMs. Sui et al. [2024] compared six formats on seven table tasks, finding HTML strongest. The ImprovingAgents benchmark (2025) tested 11 formats on flat tables, finding Markdown-KV best. Hegselmann et al. [2023] evaluated nine serializations for tabular classification. All used flat, single-sheet tables.

Spreadsheet-specific encodings. Dong et al. [2024] proposed SheetCompressor (25× compression, 78.9% F1 on table detection) and a cell-address markdown encoding that tags each cell with its A1 address. SheetCompressor replaces cell values with type labels, a design choice that we show is incompatible with QA.

Spreadsheet benchmarks. FRTR-Bench [Gulati et al., 2025], SpreadsheetBench [Ma et al., 2024], MiMoTable [Li et al., 2025], and NL2Formula [Zhao et al., 2024] provide evaluation corpora spanning enterprise workbooks to Wikipedia-derived tables. None study serialization format as an independent variable.

4 Experimental Setup

4.1 Baseline formats

We compare Ream against 10 baseline formats spanning six families (Table 1). Appendix A shows each format on the same data.

4.2 Corpora

FRTR-Bench [Gulati et al., 2025]: 30 enterprise workbooks. We generate 1,403 QA pairs across 13 question types from sheets with ≤1,000 rows (zero truncation) and sample 200. The 113 SpreadsheetBench tasks [Ma et al., 2024] are included.

Table 2: Evaluation corpora spanning the complexity spectrum.

Corpus	n	Sheets/wb	Avg. rows	Question origin	Table format
FRTR-Bench	200	2–5	200–1,000	Programmatic	XLSX
SpreadsheetBench	113	1	10–100	Forum posts	XLSX
MiMoTable	200	1	<100	Human-authored	XLSX
NL2Formula	200	1	∼10	Formula-derived	JSON→XLSX

Table 3: FRTR-Bench accuracy (%) on 200 enterprise questions. "Err" = context-length failures on GPT-4o-mini. Sorted by GPT-5.4.

		GPT-5.4	5.4-nano	GPT-4o-mini	
	Format	Acc	Acc	Acc	Err
1	**Ream**	**92.0**	**78.5**	**69.0**	**0**
1	Cell-Address MD	**92.0**	75.5	67.0	0
3	Markdown-KV	91.5	77.0	69.1	6
4	JSON	89.5	78.0	73.9†	58
5	XML	89.4	73.0	63.7†	109
6	Reverse-Idx Val	85.5	66.5	65.0	0
7	HTML	84.5	68.0	64.6	19
8	CSV	84.0	66.5	63.0	0
9	Markdown	83.0	70.0	62.5	0
10	Pandas	80.0	67.0	63.8	1
11	Raw OOXML	61.6	24.8†	32.8†	136

†Accuracy over answerable questions only.

MiMoTable [Li et al., 2025]: 200 Lookup/Compare questions over real XLSX files.

NL2Formula [Zhao et al., 2024]: 200 formula-derived QA pairs; tables converted from JSON to XLSX.

4.3 Models and scoring

We evaluate on **GPT-4o-mini** (128K context), **GPT-5.4-nano** (128K), and **GPT-5.4** (1M), all at temperature 0. Scoring: 5% numeric tolerance, case-insensitive string match with containment, boolean equivalence. Total: 11 formats × 3 models × 3 corpora × 200 questions = **19,800 planned evaluations**; 19,000+ completed.

5 Results

5.1 Enterprise workbooks (FRTR-Bench)

Ream ties Cell-Address MD at 92.0% on GPT-5.4 and leads outright on GPT-5.4-nano (78.5%) and GPT-4o-mini (69.0%). It achieves this at 45K average tokens—26% fewer than Cell-Address MD (61K) and 53% fewer than JSON (96K). Critically, Ream is the **only top-tier format with zero context failures** across all three models.

Table 4: Accuracy (%) across all three corpora. Sorted by GPT-5.4 average.

	GPT-5.4				GPT-5.4-nano			
Format	**FRTR**	**MiMo**	**NL2F**	**Avg**	**FRTR**	**MiMo**	**NL2F**	**Avg**
Ream	**92.0**	66.5	78.5	**79.0**	**78.5**	61.5	72.5	**70.8**
JSON	89.5	67.5	79.0	78.7	78.0	60.5	77.5	72.0
XML	89.4	66.5	78.5	78.1	73.0	61.0	77.0	70.3
Cell-Address MD	92.0	67.5	75.5	78.3	75.5	61.5	73.0	70.0
Markdown-KV	91.5	54.0	78.5	74.7	77.0	49.5	74.0	66.8
HTML	84.5	66.5	78.0	76.3	68.0	61.5	76.0	68.5
CSV	84.0	66.0	76.5	75.5	66.5	60.0	73.5	66.7
Markdown	83.0	66.5	76.5	75.3	70.0	64.0	75.0	69.7
Pandas	80.0	67.5	77.5	75.0	67.0	60.0	74.5	67.2
Reverse-Idx Val	85.5	66.0	76.5	76.0	66.5	52.0	63.5	60.7
Raw OOXML	61.6	66.5	78.5	68.9	24.8	58.0	76.5	53.1
Spread	30	14	4		54	14	14	

5.2 Single-sheet tables (MiMoTable and NL2Formula)

On MiMoTable (200 Lookup/Compare questions, avg. 1.7K tokens), format differences compress to 2 pp among the top 10 formats (64–68% on GPT-5.4). On NL2Formula (200 formula-derived QA pairs, avg. 0.5K tokens), the spread is 3.5 pp (75.5–79.0%). These tiny tables do not stress models enough to differentiate formats—any value-preserving format suffices.

5.3 Cross-corpus synthesis

Ream achieves the **highest cross-corpus average on GPT-5.4** (79.0%), edging JSON (78.7%) while using half the tokens. On GPT-5.4-nano, Ream leads on FRTR (78.5%) but trails JSON on NL2Formula (72.5% vs. 77.5%), reflecting JSON's advantage on tiny tables where token overhead is irrelevant.

The format spread tells the deeper story: 30 pp on enterprise workbooks (GPT-5.4), 54 pp on the smaller model, but only 4 pp on tiny tables. Format choice is a first-order design decision for complex workbooks and increasingly important for smaller models.

6 Analysis

Why does Ream excel on enterprise workbooks? Three mechanisms contribute. (i) *Explicit addressing*: the `B=`, `D=` column prefixes and absolute row numbers eliminate positional counting errors that plague CSV and Markdown on wide rows. (ii) *Token efficiency*: at 45K average tokens on FRTR, Ream leaves ample context for reasoning, while JSON (96K) and XML (137K) crowd it out. (iii) *Sheet directives*: `#!SHEET` boundaries help the model navigate multi-sheet workbooks that flat formats encode as undifferentiated text.

Why do formats converge on small tables? MiMoTable and NL2Formula tables average 1–4K tokens. At this scale, every format fits easily within the context window, positional counting rarely fails (tables have 3–6 columns), and sheet structure is trivial (single sheet). The design advantages of Ream's addressing become measurable only when the data is complex enough to need them.

Why does JSON trail Ream on FRTR but lead on NL2Formula? JSON provides explicit column-letter keys (`{"B": "West"}`) that match model pre-training. On tiny NL2Formula tables (0.5–1K tokens), this is pure benefit with no token penalty. On 1,000-row enterprise workbooks, JSON's syntactic overhead (braces, quotes, commas, indentation) balloons to 96K tokens—twice Ream's 45K—causing 58 context failures on GPT-4o-mini and degraded attention on larger models.

Smaller models benefit more from good formatting. The FRTR spread widens from 30 pp on GPT-5.4 to 54 pp on GPT-5.4-nano, and Ream's lead over CSV grows from 8 pp to 12 pp. Less capable models have less headroom for compensating when the format is suboptimal.

7 Practical Recommendations

1. **Multi-sheet workbooks**: Use Ream. It ties the best format on accuracy and uses the fewest tokens among top-tier formats.
2. **Context-constrained models**: Use Ream. It is the only top-tier format with zero context failures across all models and corpora.
3. **Simple single-sheet tables**: Any value-preserving format works. Use CSV for minimum tokens.
4. **Cost-sensitive pipelines**: Ream uses 45K avg. tokens vs. JSON's 96K on enterprise data, halving per-query API cost.

8 Limitations

1. **OpenAI models only**. Rankings may differ on Claude, Gemini, or open-weight models.
2. **Ground-truth quality**. FRTR questions are programmatic; MiMoTable answers are extracted from verbose prose.
3. **Values-only evaluation**. Ream's formula encoding (`==R[-1]C*0.6`), defined names, and table references are not exercised in this benchmark.
4. **Sample size**. 200 questions per corpus; 95%-confidence intervals $\approx \pm 4\%$.

9 Conclusion

We introduced Ream, a sparse text format for serializing spreadsheet workbooks for LLM consumption, and benchmarked it against 10 alternative formats across four corpora and three models. Ream achieves the highest cross-corpus accuracy (79.0% on GPT-5.4) while using 26–53% fewer tokens than its nearest competitors, and is the only top-tier format that never exceeds context limits. The key insight is that **format choice is a first-order design decision for complex spreadsheets**: on enterprise workbooks, the gap between the best and worst format exceeds 30 percentage points.

We release the Ream specification, evaluation harness, and all results at https://github.com/blader/ream.

References

H. Dong, S. Zhao, Y. Tian, D. Xiong, S. Xia, M. Zhou, D. Lin, J. Cambronero, J. He, H. Han, and D. Zhang. SpreadsheetLLM: Encoding spreadsheets for large language models. In *Proc. EMNLP*, 2024.

A. Gulati, A. Sen, S. Sarguroh, and R. Paul. From rows to reasoning: A retrieval-augmented multimodal framework for spreadsheet understanding. *arXiv preprint arXiv:2601.08741*, 2025.

S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag. TabLLM: Few-shot classification of tabular data with large language models. In *Proc. AISTATS*, 2023.

G. Li, Z. Du, W. Zheng, and S. Song. MiMoTable: A multi-scale spreadsheet benchmark with meta operations for table reasoning. In *Proc. COLING*, 2025.

Z. Ma, W. Zhang, J. Zhang, C. Yu, T. Zhang, G. Zhang, Y. Luo, J. Wang, and J. Tang. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. In *NeurIPS Datasets and Benchmarks Track*, 2024.

Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang. Table meets LLM: Can large language models understand structured table data? In *Proc. WSDM*, 2024.

W. Zhao, Z. Hou, S. Wu, Y. Gao, H. Dong, Y. Wan, H. Zhang, Y. Sui, and H. Zhang. NL2Formula: Generating spreadsheet formulas from natural language queries. In *Findings of EACL*, 2024.

A Format Samples

All samples encode the same Revenue sheet (header plus one data row).

A.1 Ream

```
#!REAM 9
#!SHEET Revenue
#!HEADERS 1:1
2 | Date | Region | SKU | Qty | UnitPrice | Revenue |
3 | 2024-01-01 | West | SKU-001 | 195 | 49.554 | 9663.05 |
```

A.2 CSV

```
=== Sheet: Revenue ===
Date,Region,SKU,Qty,UnitPrice,Revenue
2024-01-01,West,SKU-001,195,49.554,9663.05
```

A.3 Markdown

```
## Sheet: Revenue
| Date | Region | SKU | Qty | UnitPrice | Revenue |
|------|--------|-----|-----|-----------|---------|
| 2024-01-01 | West | SKU-001 | 195 | 49.554 | 9663.05 |
```

A.4 Markdown-KV

```
# Sheet: Revenue
## Row 3
Date: 2024-01-01
Region: West
SKU: SKU-001
```

```
Qty: 195
UnitPrice: 49.554
Revenue: 9663.05
```

A.5 JSON

```
{"sheets": {"Revenue": {"rows": [
  {"A": "Date", "B": "Region", ..., "_row": 2},
  {"A": "2024-01-01", "B": "West", "C": "SKU-001",
   "D": "195", "E": "49.554", "F": "9663.05", "_row": 3}
]}}}
```

A.6 HTML

```
<h3>Revenue</h3>
<table>
  <tr><th>Date</th><th>Region</th><th>SKU</th><th>Qty</th>...</tr>
  <tr><td>2024-01-01</td><td>West</td><td>SKU-001</td><td>195</td>...</tr>
</table>
```

A.7 XML

```
<sheet name="Revenue">
  <row n="3">
    <Date>2024-01-01</Date>
    <Region>West</Region>
    <SKU>SKU-001</SKU>
    <Qty>195</Qty>
  </row>
</sheet>
```

A.8 Cell-Address Markdown

```
[Sheet: Revenue]
A2,Date|B2,Region|C2,SKU|D2,Qty|E2,UnitPrice|F2,Revenue
A3,2024-01-01|B3,West|C3,SKU-001|D3,195|E3,49.554|F3,9663.05
```

A.9 Pandas

```
Sheet: Revenue
         Date Region      SKU  Qty  UnitPrice   Revenue
0  2024-01-01   West  SKU-001  195     49.554  9663.05
```

A.10 Reverse-Index Values

```
{"Revenue": {
  "Date": "A2", "Region": "B2", "SKU": "C2",
  "2024-01-01": "A3", "West": "B3", "SKU-001": "C3",
  "195": "D3", "49.554": "E3", "9663.05": "F3"
}}
```

A.11 Raw OOXML

```
[File: xl/worksheets/sheet2.xml]
<?xml version="1.0"?>
<worksheet xmlns="http://schemas.openxmlformats.org/...">
<sheetData>
  <row r="3"><c r="A3" t="s"><v>42</v></c>...</row>
</sheetData></worksheet>
```